

Bloque B4

Validación y manejo de errores

Laboratorio. Validar la entrada con un middleware y centralizar los errores en un solo lugar. Duración aproximada: 2 horas.

Por qué validar SIEMPRE en el servidor

El cliente puede enviar cualquier cosa: campos vacíos, tipos incorrectos, o nada. Nunca confíes en que el body viene bien formado. Aunque tu formulario HTML valide, alguien puede llamar tu API directamente (con curl, Postman, un script). La validación del servidor es la única que realmente protege tus datos.

Códigos de estado relevantes para validación:

Código	Significado	Cuándo usarlo
400	Bad Request	El cliente mandó algo mal formado o incompleto
401	Unauthorized	Falta o falla la credencial (lo maneja verificarToken)
404	Not Found	El recurso pedido no existe
500	Server Error	Algo falló del lado del servidor (no culpa del cliente)

Fragmento 1 — Middleware de validación

Archivo: servicio-recurso/src/validarTarea.js

Sacamos la validación a su propio middleware reutilizable. Verifica que `titulo` exista, no esté vacío y no exceda el límite. Si algo falla, responde **400** y corta la cadena (no llama `next()`). Si todo está bien, llama `next()` para seguir a la ruta.

```
function validarTarea(req, res, next) {
  const { titulo } = req.body;
  if (titulo === undefined || titulo === null || String(titulo).trim() === '') {
    return res.status(400).json({ error: 'El campo "titulo" es obligatorio' });
  }
  if (String(titulo).length > 200) {
    return res.status(400).json({ error: 'El "titulo" no puede exceder 200 caracteres' });
  }
  next();
}

module.exports = validarTarea;
```

Fragmento 2 — Aplicar la validación a rutas específicas

Archivo: servicio-recurso/src/tareas.routes.js

Un middleware no tiene que ser global. Puedes aplicarlo solo a las rutas que lo necesitan, pasándolo como argumento antes del manejador. Aquí `validarTarea` corre antes de crear y de actualizar, pero no en GET ni DELETE (que no reciben body con título).

```
const validarTarea = require('./validarTarea');

router.post('/tareas', validarTarea, async (req, res, next) => { /* ... */ });
router.put('/tareas/:id', validarTarea, async (req, res, next) => { /* ... */ });
```

Manejo centralizado de errores

Hasta ahora cada ruta hace `next(err)` en su `catch`. ¿A dónde va ese error? A un middleware especial que Express reconoce porque tiene **cuatro** argumentos: `(err, req, res, next)`. Esa firma es lo que lo distingue de un middleware normal.

Centralizar tiene dos ventajas: (1) un solo lugar decide cómo se ve un error 500, y (2) las rutas quedan limpias, sin repetir manejo de errores en cada una.

Fragmento 3 — El manejador de errores (4 argumentos)

Archivo: `servicio-recurso/src/index.js`

Va **al final**, después de todas las rutas. Registra el detalle completo en el log del servidor (para que tú lo veas) pero devuelve al cliente un mensaje genérico. **Nunca** mandes el objeto de error de MySQL al cliente: expondría nombres de tablas, columnas y fragmentos de SQL — una fuga de información.

```
// AL FINAL del index.js, despues de las rutas
app.use((err, req, res, next) => {
  console.error('[RECURSO][ERROR]', err); // detalle -> log servidor
  res.status(500).json({ error: 'Error interno del servidor' }); // generico -> cliente
});
```

Mala práctica que debes evitar

Escribir `res.status(500).send(err)` en cada ruta. Eso filtra detalles internos al cliente y repite código. El patrón correcto es: la ruta solo hace `next(err)`; el manejador central decide el resto. Aplica este mismo manejador en AMBOS servicios (A y B).

Paso de prueba

Con el Servicio B corriendo y un token válido:

1. POST :3002/tareas con body vacío `{}` → debe responder **400** 'titulo es obligatorio'.
2. POST :3002/tareas con un título normal → **201**.
3. Verifica que la forma del error siempre es `{ "error": "... " }`, consistente con lo que diseñaste en B0.

Resultado esperado de este bloque

- La validación vive en su propio middleware y se aplica solo donde hace falta.
- Los errores se manejan en un único lugar por servicio, con la firma de 4 argumentos.
- El cliente nunca recibe detalles internos; el servidor los registra en su log.
- La forma de error es consistente en toda la API.